D&D AI Battle Map Generator

User manual. Written for somebody who has never opened ComfyUI and does not intend to learn what a diffusion model is. Nothing here assumes any of that.

> **In one sentence.** You type "*a harbour with one large moored ship*", the program works out the floor plan, draws it, and then paints a finished top-down battle map you can load into Roll20 or Foundry and play on tonight.

Contents

1. What this is, and what it is not

This program makes **battle maps**: the top-down picture of a place that a group fights across, one square per five feet. It does not make character art, region maps of a kingdom, or illustrations.

Three things are deliberately never drawn

Not drawn	Why
Any text	Names, numbers and labels ruin a map at the table. Area names are stored in the plan file and shown in the preview, never painted into the image.
Living creatures	The GM puts tokens on the map. A painted-in guard is a guard you can never move or kill.
Grid lines	Roll20 and Foundry draw their own grid at their own scale. A baked-in grid never lines up with it.

You still choose the size *in squares* — that is how the proportions and the detail level are decided. Only the visible lines are absent.

The two stages

Everything the program does falls into one of two halves, and knowing which half went wrong is most of troubleshooting.

1. **The plan.** Your sentence becomes a description of the place, and that description becomes a floor plan: rooms, corridors, walls, doors, water, props. Fast, and on the processor.
2. **The painting.** The plan is turned into a written instruction for the image model, which paints the finished map. Slow, and on the graphics card.

The plan is a file you can look at, edit and keep. If the map came out wrong, look at the plan first: **what you see in the Editor is what gets painted**.

Who writes what

Three different things do three different jobs, and it is worth knowing which is which.

Step	Who does it	What comes out
1. Describe the place	**Ollama**, an **AI agent**, or **you**	A written description: what kind of place, which areas it has, what it is built of, what is lying about
2. Lay it out	The program itself	The exact floor plan - the picture you edit on the Editor tab
3. Write the painter's instructions	The program itself	A structured caption: every wall, door and object with its coordinates, wrapped in the wording from step 1
4. Paint it	Ideogram 4 in ComfyUI	The finished map

Note what the language model does **not** do. It never places anything and never computes a coordinate - that is step 2, and it is ordinary arithmetic, which is why the layouts are always valid. And it does not write the painter's instructions either: step 3 builds those from the plan itself, so the shape of the map cannot drift.

What the language model contributes is **words**. Your one line becomes several paragraphs of concrete detail - wet cobbles with tar stains rather than "stone", a harbourmaster's office and a cargo warehouse rather than "some rooms" - and that wording is what step 3 wraps around the coordinates. That is the whole of its job, and it is the difference between a thin map and a rich one.

2. What has to be running before you start

The program does not paint anything itself. It drives two services on your own computer. Nothing is uploaded anywhere and no account is needed.

Service	What it does	Required	Address
ComfyUI	paints the picture	yes	`http://127.0.0.1:8188`
Ollama	writes the description the map is built around	no	`http://127.0.0.1:11434`

What Ollama is for, and how to do without it

Ollama writes step 1 above: it turns your one line into the detailed description the painter's instructions are built around. Nothing else in the program needs it.

So there are three ways to get that description, and they are interchangeable:

Who describes the place	How	Needs Ollama
Ollama, locally	Create tab, or `pipeline.py plan`	yes
An AI agent	It writes the description itself and calls `agent_api.generate({...})`	no
You	Write a fuller scene description yourself, or build the plan with **Blueprint only** and shape it in the Editor	no

An AI agent replaces Ollama completely. Any capable model - the assistant you are already talking to, or anything else that can run Python - can write the description itself and hand it straight to the program. It is faster than a local model, it leaves the whole graphics card free for painting, and it gives finer control over the scene. Everything an agent needs ships with the program: `AGENTS.md` next to the executable documents every field, and `tools/agent_api.py` is the entry point. See chapter 12.

Without any of them the program still works. Blueprint only builds a floor plan instantly with no language model at all, and the Editor lets you build a map square by square. What you lose is only the enrichment: the painter is then told what your own sentence said, plus the materials the chosen style always contributes.

They take turns with the graphics card. The planner and the painter are both far too big to sit in video memory together, so the program hands the card over automatically: before a render it asks Ollama to drop its model, and before planning it asks ComfyUI to drop its own. You do not have to do anything - it just means the first few seconds of each step are spent handing over.

Model files ComfyUI needs

Four files, about 27.5 GB together. Each goes in a specific folder inside your ComfyUI installation:

File	Size	Folder
`ideogram4_fp8_scaled.safetensors`	8.6 GB	`models/diffusion_models/`
`ideogram4_unconditional_fp8_scaled.safetensors`	8.6 GB	`models/diffusion_models/`
`qwen3vl_8b_fp8_scaled.safetensors`	9.9 GB	`models/text_encoders/`
`flux2-vae.safetensors`	0.3 GB	`models/vae/`

Both `ideogram4` files are needed. One is the ordinary model, the other is its unconditional twin, and the program uses the difference between them to sharpen the result. With only the first file the picture comes out pale, vague and low in contrast. This is the single most common installation mistake.

What the computer needs

	Minimum	Comfortable
Graphics card	NVIDIA, 8 GB video memory	NVIDIA, 12 GB or more
System memory	32 GB	64 GB
Free disk space	35 GB	50 GB
Windows	10, 64-bit	11, 64-bit

System memory matters more than usual here: the models are far too big to sit in video memory together, so most of them live in ordinary RAM and are streamed to the card as needed. With 8 GB of video memory, start ComfyUI with the `--lowvram` option if it struggles.

A render takes minutes, not seconds. The first one after starting ComfyUI is noticeably slower than the rest, because 27 GB of model files have to be read off the disk.

3. Quick start: your first map

1. Start ComfyUI and wait until its page opens in a browser.
2. Start `DndBattlemapGenerator.exe`.
3. Open the **Settings** tab and press **Test both connections**. The ComfyUI line must turn green and say *ready*. If it does not, jump to chapter 13 — nothing else will work until it does.
4. Open the **Create** tab and type a scene, for example:

> `a flooded crypt with cracked sarcophagi and standing water`

5. Pick a look from the style cards. For this example, *Gothic Crypt & Catacombs*.
6. Leave the size at **Medium** for now.
7. Press **MAKE MY BATTLE MAP** and wait.

The plan appears within seconds and the painted map after a few minutes. Both are saved into the `output` folder, in a subfolder named after the map.

> **If you do not want to wait.** The **Blueprint only (instant, no AI)** button builds just the floor plan — instantly, and without the language model. Useful when you want to try several layouts before spending time on painting one.

4. The Create tab — describing the place

The description

One or two sentences. Two things are worth naming: **what the place is**, and **what it is made of**.

```
Weak:   a dungeon
Better: a damp stone dungeon with a collapsed corridor and a flooded lower room
```

You do not need to describe the geometry. Do not write "three rooms in a row" — the program decides the arrangement. Describe the place, not the floor plan.

Size in squares

Two sliders, **Width** and **Height**, from 10 to 150 squares each, with the size in feet shown underneath. Next to them are quick buttons:

Button	Squares	When to use it
Small	17 x 13	A single room, a short skirmish
Medium	25 x 19	An ordinary fight — take this if unsure
Large	66 x 50	A big scene: a harbour, several buildings
Huge	100 x 75	A whole district or dungeon level
Giant	150 x 150	The maximum. Noticeably slower to paint

One square is the standard five feet (1.5 m) of D&D.

The bleed margin

Around whatever size you choose, the program adds an empty border two squares wide. In the Editor it is drawn hatched and dimmed, outside the gold line that marks your field, and you cannot paint in it.

It is added, never taken away. Ask for 25 x 19 and you get 25 x 19 squares to work with; the picture that comes out is 29 x 23 with a blank rim.

The reason is simple: image models are least reliable at the very edge of a picture, where they run out of context and start inventing. Given a margin, whatever goes wrong goes wrong in empty space instead of eating the corner of a room. It also matches how a printed battle map looks, with its content inset from the paper edge, so the result reads as a finished sheet rather than something cropped.

Layout — the shape of the scene

Leave it on *(from style)* and the style decides. Override it when you want something specific:

Layout	Builds
dungeon	Separate rooms joined by corridors
building	One structure, rooms divided by internal walls
cavern	An organic cave with irregular walls
open	Open ground with no enclosing walls: a field, a crossroads, a camp
forest	Dense woodland; clearings are cut out of the thicket, paths between them
swamp	Standing water, reed beds, islands of dry land, plank walkways
ruins	Open ground strewn with fragments of collapsed walls
street	A city block with buildings along a road, open ground beyond them
district	Wall-to-wall city: buildings edge to edge with alleys between, no open country in sight
deck	One ship under way, its deck filling the map, open water all round
arena	One dramatic chamber
harbour	A quay, open water and a moored ship

Style — the look

Sets materials, palette and lighting. Styles are shown as a grid of cards, each painted in that style's own colours, so the right one can be spotted without reading every name. Hovering a card shows its description.

Ground clutter and prop density

Terrain scatters water, pits, rubble or undergrowth across the floor, and **amount** says how much. **Prop density** controls how many objects — barrels, crates, tables — are placed. High density on a large map means a lot of furniture; if the result feels cluttered, lower it and re-plan.

> **If waiting is not for you. Blueprint only (instant, no AI)** builds only the plan — instantly and with no language model. Handy for flipping through layout options before spending time on a render.

5. The Editor tab — changing it by hand

The plan is shown as an editable grid. Everything you see here is what will be painted, so this tab is both the place to fix mistakes and the place to build a map from nothing.

Left mouse uses the current tool. **Right or middle mouse dragged** moves the view. **Wheel** zooms.

Tools

Each has its own icon next to its name, and a tooltip when you hover it.

- **Look around** — look only, change nothing. The safe mode.
- **Brush** — paints the selected material. Brush size is the slider below.
- **Rectangle** — fills a rectangle: press, drag, release.
- **Place prop** — places an object from the catalogue, or one of your own.
- **Erase prop** — removes an object.
- **Custom area** — a rectangle you describe in your own words.
- **Effect** — lays an effect over an area: fire, fog, fireflies.

Materials

Material	Meaning
floor	Ground you walk on
wall	Solid wall, blocks movement and sight
door	Door — belongs inside a wall
window	Window — an opening in a wall. Light and sight pass, a body does not
water	Water
pit	Pit or drop
rubble	Loose rubble, difficult ground
vegetation	Undergrowth and bushes
bridge	Timber walkway or ship decking
stairs	Stairs
void	Emptiness — the eraser, clears everything

The prop catalogue

Objects are chosen from a scrolling grid of icons with names. The tooltip says what the thing is and how it plays — for example *full cover* or *difficult ground*.

Your own object

Tick **Custom object** in the **Place prop** tool. You give a name and a description, then choose how far the AI may embellish it:

- **No — exactly as written** — draw precisely that and add nothing.
- **A little** — fill in fitting detail.
- **Freely** — take it as a starting point and develop it.

> **Что стоит знать про свои области.** Прямоугольник — это всегда «нарисуй здесь вот это». Прямоугольника со значением «здесь ничего не должно быть» не существует: если обвести пустой пол, художник построит на этой рамке комнату. И если обвести стены слева, справа, сверху и снизу, четыре рамки сложатся в пятую стену внутри помещения. Обводите то, что стоит в одном конкретном месте — фонтан, лестницу, решётку, конкретную дверь. Всё, что просто стоит вдоль стены, лучше описать словами в описании самой комнаты.

Custom area — an exact description for an exact place

The **Custom area** tool. Drag a rectangle and describe what belongs there: a barred gate, a breach in the wall, a particular door. This goes into the instructions for the painter at **top priority** together with its coordinates, so the thing ends up exactly where you marked it.

This is also how you get non-standard walls and doors when the material list has no word for what you want.

Hovering and right-clicking

Hover any square and a tooltip names what stands on it — an object, an effect, one of your areas, or simply the floor material — along with the square's coordinates.

Press the **right mouse button** and a menu opens for whatever is under the cursor. For your own objects, effects and areas you can change the name, the description and the amount of embellishment right there, or delete the thing. On an empty square it offers to pick that material into the brush.

Right-click *and drag* still moves the view — the menu only opens on a click that stayed in one place.

Effects — a layer of their own

The **Effect** tool. Drag a rectangle and choose an effect from the icon grid: fire, embers, smoke, fog, mist, fireflies, arcane glow, holy light, poison gas, blood, ice, webs, sparks, ash, steam, shadow. You can define your own as well, exactly as with objects.

Effects are the top layer. They never change the floor and never affect whether something can be walked through: they are light, smoke and weather painted over the finished map. The **Strength** setting says how strongly the effect reads.

> **The "Rebuild walls" button.** It re-derives every wall from the floor you have painted. Press it after large changes — this is exactly how generated maps get their walls, so the result is consistent with them.

Your edits are not thrown away silently

Building a new plan replaces the current one completely. If you have changed anything by hand, the program stops and asks first, and offers to save what you have before it goes ahead:

- **Save it first, then build** — writes `map.json` and `preview.png` into the output folder, then builds the new plan.
- **Replace it without saving** — throw the edits away.
- **Keep what I have** — cancel, change nothing.

A plan you have not touched is replaced without asking — there is nothing of yours in it to lose.

Opening somebody else's plan

File → Open map.json… shows every plan in the `output` folder as a grid of thumbnails, so you see the actual layout rather than only a folder name. This is how you open a map that was built by an AI agent or from the command line, look at it, and change it.

The settings the map was made with come back with it: the scene description, the style, the size, the layout and the ground clutter.

Double-clicking a card opens it immediately. Escape closes the window.

6. The Render tab — painting settings

This tab paints whatever is currently in the Editor, so an edited plan can be re-rendered without being re-planned.

Setting	What it does
Preset	*Turbo* 12 steps - fast and rough, for trying an idea out. *Default* 20. *Quality* 48 - the one to use. *Ultra* 64 - a third again the time for a little more settling in the fine detail, worth it on a map you are keeping.
Guidance	How strictly the painter follows the instructions. Higher means more obedient and less imaginative. The default is a good compromise.
Resolution	Set as target megapixels; the actual width and height follow the shape of your grid. Bigger means slower and more detailed.
Seed	The same seed with the same plan gives the same picture. `-1` means a new random one each time.

The caption

Below the settings is the exact instruction that will be sent to the painter, in two views:

- **Readable** — the scene description, the look, the lighting, the palette as colour swatches, and every placed object listed with its coordinates. Worth reading once, to see what the program actually asks for.
- **Raw JSON** — the same thing exactly as it goes over the wire.

Press **Edit by hand** and the caption becomes yours: type whatever you like into the raw view and it is sent exactly as you leave it. The panel tells you whether what you have typed is still valid JSON. While a hand-written caption is in force the map no longer rewrites it, so editing the plan afterwards will not be reflected until you press **Reload from plan** — or **Back to automatic**, which throws your version away and returns to the caption built from the plan.

This is the sharpest tool in the program. It is also the only one that can break the rules that keep text and creatures out of your maps, so read what you are replacing.

7. The Dungeondraft tab — an editable map instead of a picture

The Render tab paints your layout. This one hands the same layout to **Dungeondraft** instead, as a real `.dungeondraft_map` file built out of the asset packs you already own. Nothing is painted and no GPU is used; it takes a second or two. Open the result in Dungeondraft and every wall, door, barrel and puddle is a separate object you can drag, delete or recolour.

Use it when you want to keep editing the map. Use the Render tab when you want a finished picture. They read the same plan, so you can do both from one description.

Before the first export: build the asset library

The bottom half of the tab is about the library, and it has to exist before an export can choose anything. **Scan & Index Asset Packs** reads Dungeondraft's own `Dungeondraft.pck` and every custom `.dungeondraft_pack` in the folder named in Settings, and writes an index into `data/assets.db`. It uses every core you have and takes a few minutes for a large collection. Do it once, and again whenever you install new packs.

The counters above the buttons say what is in the index: packs, how many are enabled, textures, and how many have been described by the vision model.

Vision tagging, and why it is optional

Indexing knows a texture's name, size and colours. It does not know that `obj_1042.png` is a butter churn. The tagging buttons hand each thumbnail to a local Ollama vision model and record what it is, which is what lets a plan asking for a "churn" find one.

It is slow — seconds per asset, across tens of thousands — so it is split up: **Check Quality on 200 Assets First** to see whether the model is any good on your library before committing to the full pass, then stock only, custom only, or everything. Exporting works without it; the matcher falls back to reading file names, and the report tells you how many props were chosen that way.

Exporting

The top of the tab shows the layout that is loaded — style, grid, rooms, props — and three settings:

- **Output file.** Where the `.dungeondraft_map` goes.
- **Random variation seed.** Which asset gets picked where the library offers several. The same seed gives the same map every time.
- **Auto-render missing props via Foundry.** When the library has nothing for something the plan asked for, generate it with Ideogram, cut the background out, and add it to a custom pack of your own. Needs ComfyUI, and it is the slow option on this tab.
- **Launch in Dungeondraft when done.** Opens the file in Dungeondraft, if the path to it is set in Settings.

After an export the tab reports what it placed: walls, objects, doors, how many packs the map needs, and how many props were matched from the catalogue against how many from the file name alone. That last pair is the honest quality number — a map matched entirely by file name will have things in roughly the right places that are not quite the right things.

What the export does with your layout

Walls are traced from the wall squares of the plan and drawn along the grid lines, so a run of wall is one continuous line, corners actually meet, and no square is left cut in half with nowhere for a token to stand. Doorways become real portals cut into that wall rather than gaps beside it. Each body of water is one polygon following its shoreline. Floors get a tileset, decks and gangways get planking, and the ground under undergrowth, rubble and lake beds is painted with its own terrain rather than one texture everywhere.

The empty margin around the field is not exported. It is there so the painter makes its mistakes off the edge of your map; Dungeondraft has no such problem, so the file opens at exactly the size you planned.

A map records which packs it needs. If you send the file to somebody who does not own them, Dungeondraft will open the map and tell them what is missing.

8. The Styles tab — the look library

Every style is a single file in the `styles` folder. There are thirty-two of them, and this tab edits them in place: materials, palette, lighting, aesthetics, default layout and the standard set of props.

Kind of place	Styles
Settlements	Village & Hamlet, Farmstead & Barn
Towns and cities	City Streets & Market, City Harbour & Docks, Cyberpunk Neon Alley
Fortresses	Castle Keep & Courtyard
Dungeons	Classic Stone Dungeon, Gothic Crypt & Catacombs, Sunken Ancient Temple, Sewer Tunnels
Underground	Underdark Crystal Cavern, Abandoned Mine, Volcanic Dragon Lair
Sacred places	Grand Temple, Graveyard & Mausoleum
Interiors	Cozy Medieval Tavern, Arcane Library & Study
Wilderness	Enchanted Forest Clearing, Marsh & Peat Bog, Mountain Road & Bridge, Frozen Pass, Desert Ruins, Coastal Sea Cliff Ruins, Bandit Camp
At sea	Ship's Deck at Sea
Science fiction	Sci-Fi Derelict Starship

The shared part that every style inherits lives in `styles/_base.json`. Among other things it holds the one sentence that forbids text, creatures and grid lines.

> **Be careful with the base file.** The image model has no way of being told "do not draw X" — that sentence is the only thing keeping labels and people out of your maps. Rewrite it and they come back.

To make your own style, copy any style file, change the `id` inside, and it appears in the program after a restart.

9. The Settings tab — services and models

Setting	Notes
ComfyUI address	Normally `http://127.0.0.1:8188`
Ollama address	Normally `http://127.0.0.1:11434`
Model filenames	Must match the files in `ComfyUI/models/` exactly, character for character
Planner model	The name as `ollama list` shows it
Temperature	How inventive the planner is. Higher means more variety and less predictability
Vision model	The Ollama model that describes Dungeondraft asset thumbnails during tagging. It has to be a model with vision — the app checks and says so if it is not
Custom assets folder	Where your `.dungeondraft_pack` files live, so the indexer can find them
Dungeondraft executable	Optional. Set it and the Dungeondraft tab can open a finished map for you

Test both connections checks both services and reports what it found. Everything is saved to `config.json` beside the program and shared with the command line tools.

There is one more tab, **Docs**, which holds a short reminder of what each tab is for and the handful of rules worth keeping in mind. This manual is the long version of it.

10. What lands in the output folder

Everything belonging to one map goes into `output/<name>/`:

File	What it is
`battlemap*.png`	**The finished map.** This is the one you load into your VTT
`*.dungeondraft_map`	**The editable map**, if you exported one. Open it in Dungeondraft
`*.report.json`	What that export placed: walls, doors, objects, and which asset packs the map needs
`preview.png`	The labelled plan, for your eyes only — never painted
`map.json`	The plan itself. Can be reopened and re-rendered at any time
`caption.json`	The exact instructions the painter was given
`spec.json`	The design description the first stage produced

Keep `map.json`. As long as you have it you can re-render the same place at a different size, in a different style, with a different seed, for as long as you like. The PNG is a result; the plan is the source.

11. Working from the command line

Needs Python 3.10 or newer and the Pillow package:

```
pip install pillow
```

List everything available:

```
python tools/pipeline.py styles
```

Plan and paint in one go:

```
python tools/pipeline.py auto "a harbour with one large moored ship" --style city_harbour
```

Plan only, then paint later:

```
python tools/pipeline.py plan "flooded crypt with sarcophagi" --style gothic_crypt
python tools/pipeline.py generate output/flooded_crypt/map.json
```

Redraw the human preview, or check and repair a plan:

```
python tools/pipeline.py preview output/flooded_crypt/map.json
python tools/pipeline.py validate output/flooded_crypt/map.json
```

Assemble the same plan as an editable Dungeondraft map instead of painting it (chapter 7), or print the instructions the painter would be given without painting anything:

```
python tools/pipeline.py dungeondraft output/flooded_crypt/map.json
python tools/pipeline.py caption output/flooded_crypt/map.json
```

Flag	Meaning
`--cols`, `--rows`	Exact grid size in squares
`--size`	`small` … `giant`
`--seed`	Fixed seed, so the same plan comes out again
`--out`	Where to write the result
`--sketch`	Hand the planner a rough layout picture. Needs an Ollama model that can see images

12. Automation for AI agents

A capable AI agent does not need Ollama at all: it writes the design description itself, which is faster and leaves the graphics card free for painting.

```
import sys; sys.path.insert(0, "tools")
import agent_api

result = agent_api.generate({
    "title": "Baldur's Gate Docks",
    "style": "city_harbour",
    "layout": "harbour",
    "grid": {"cols": 40, "rows": 30},
    "scene_summary": "A stone quay with one large ship moored alongside and a gangway ashore.",
    "rooms": [
        {"label": "Cargo Warehouse", "size": "l", "props": ["crate", "barrel", "cart"]},
        {"label": "Harbourmaster",   "size": "s", "props": ["table", "bookshelf"]},
    ],
})

print(result["images"])
```

The full description of every field is in `AGENTS.md`, which ships next to the program.

> **The rule that matters.** Describe the place; never compute coordinates. All geometry — where rooms sit, how corridors run, where walls and doors go — is decided by the program. That split is why the layouts are always valid.

13. If something goes wrong

Symptom	What to do
The status line says `ComfyUI: offline`	ComfyUI is not running, or it is on a different port. Start it and open `http://127.0.0.1:8188` in a browser to check. Then compare the address in the Settings tab.
Rendering fails immediately	One of the model filenames in Settings does not match a real file in `ComfyUI/models/`. Compare them character by character.
The picture is pale, vague, low in contrast	The `ideogram4_unconditional` file is missing or misnamed. Both model files are required.
Out of memory while rendering	Close other programs that use the graphics card, start ComfyUI with `--lowvram`, and lower the target megapixels in Settings.
`Ollama: offline` although it is installed	Ollama is not running, or the model name in Settings differs from what `ollama list` shows.
Planning takes very long	The planner model is large and is competing with ComfyUI for the graphics card. Pull a smaller one, or raise the timeout.
A door on the plan came out as a plain doorway	A door only makes sense inside a wall. If there is open space beside it, the program honestly turns it into an opening. Draw a wall and put the door into it, or press **Rebuild walls**.
Something you asked for is missing	Minor decoration is left to the painter's judgement. If an object matters, place it as a **Custom object** or mark it with a **Custom area** — then it is drawn strictly in its place.
The map looks empty	Raise **Prop density** on the Create tab, or place what you want yourself in the Editor.
The painted map does not match the plan	Look at the Editor: what is there is what was sent. If the plan is right and the picture is not, raise **Guidance** on the Render tab and use **Custom areas** for the parts that must be exact.
Text appeared in the picture	Something re-introduced it into the instructions. Check the forbidding sentence in `styles/_base.json`.

The general rule. Wrong shape of the place — fix the plan in the Editor. Wrong look of the place — change the style or the Render settings. Almost everything falls into one of those two.